AGRAG: Advanced Graph-based Retrieval-Augmented Generation for LLMs

Yubo Wang[1], Haoyang Li[2*], Fei Teng[1], Lei Chen[1,3,4]
[1]The Hong Kong University of Science and Technology, Hong Kong SAR, China
[2]The Hong Kong Polytechnic University, Hong Kong SAR, China
[3]The Hong Kong University of Science and Technology (Guangzhou), China
[4]Guangzhou HKUST Fok Ying Tung Research Institute, China
ywangnx@connect.ust.hk, haoyang-comp.li@polyu.edu.hk, fteng@connect.ust.hk, leichen@cse.ust.hk

***Abstract*—Graph-based retrieval-augmented generation (Graph-based RAG) has demonstrated significant potential in enhancing Large Language Models (LLMs) with structured knowledge. However, existing methods face three critical challenges: Inaccurate Graph Construction, caused by LLM hallucination; Poor Reasoning Ability, caused by failing to generate explicit reasons telling LLM why certain chunks were selected; and Inadequate Answering, which only partially answers the query due to the inadequate LLM reasoning, making their performance lag behind NaiveRAG on certain tasks. To address these issues, we propose AGRAG, an advanced graph-based retrieval-augmented generation framework. When constructing the graph, AGRAG substitutes the widely used LLM entity extraction method with a statistics-based method, avoiding hallucination and error propagation. During retrieval, AGRAG formulates the graph reasoning procedure as the Minimum Cost Maximum Influence (MCMI) subgraph generation problem, where we try to include more nodes with high influence score, but with less involving edge cost, to make the generated reasoning paths more comprehensive. We prove this problem to be NP-hard, and propose a greedy algorithm to solve it. The MCMI subgraph generated can serve as explicit reasoning paths to tell LLM why certain chunks were retrieved, thereby making the LLM better focus on the query-related part contents of the chunks, reducing the impact of noise, and improving AGRAG's reasoning ability. Furthermore, compared with the simple tree-structured reasoning paths, our MCMI subgraph can allow more complex graph structures, such as cycles, and improve the comprehensiveness of the generated reasoning paths. The code and prompt of AGRAG are released at: https://github.com/Wyb0627/AGRAG.**

***Index Terms*—Large Language Models, RAG.**

I. Introduction

Large Language Models (LLMs) [1]–[5] have demonstrated remarkable capabilities in natural language understanding and generation [6]–[14]. However, their reliance on static, internalized knowledge often leads to factual hallucinations and limited adaptability to dynamic information [15]–[18], and retraining or finetuning the LLM can lead to high computational costs [19], [20]. Retrieval-Augmented Generation (RAG) addresses this by integrating external factual knowledge during inference, improving accuracy and enabling real-time updates. Yet, traditional RAG methods' retrieval [15], [21] is based on simplistic mechanisms, such as whole passage's dense vector similarity, making them overlook the capture of more fine-grained semantic relationships between entities [22]. To overcome these limitations, researchers have proposed various Graph-based RAG models [23]–[31]. These models first utilize LLMs to extract entities and relations from the source documents to serve as graph nodes and edges and construct a graph, and then leverage this graph-structured knowledge to model complex relationships between entities to enable structured reasoning over inter-connected information, enabling the capture of more fine-grained entity-level relationships. [22].

Currently, the Graph-based RAG models can be classified into two categories, i.e., *community-based models*, and *walking-based models*. The community-based models [23], [24], [26] first map the query to graph nodes considering their semantic similarity or keyword matching. Then, they will retrieve the mapped graph nodes' one or multi-hop neighborhood. On the other hand, with similar mapping approach from query to graph nodes, walking-based models [25], [27]–[29], [32] apply walk-based graph algorithms, such as Personalized PageRank (PPR) [33], based on the mapped graph nodes, to score graph nodes and edges based on the graph's topological structure and query information, and then retrieve Top-k text chunks based on high-scoring graph nodes, capturing the fine-grained entity level relationships.

However, on complex QA tasks that require knowledge summarization or content generation [34], the existing RAG models cannot answer these complex queries due to:

1) **Inaccurate Graph Construction:** Current Graph-based RAG models apply LLM for entity and relation extraction to construct graphs. However, the LLM can inevitably suffer from the hallucination issue [15]–[18], leading to inaccurate entity extraction, introducing noise into the constructed graph and propagate through the whole pipeline, affecting the model effectiveness.
2) **Poor Reasoning Ability:** The community-based models' retrieval only focuses on a narrow graph neighborhood and misses the graph's long-hop dependency, which is crucial for tasks that require multi-hop reasoning. In contrast, walking-based models do account for these long-hop dependencies during retrieval; however, they simply feed the retrieved text chunks directly into the LLM as contexts. Consequently, both of their retrieval and reasoning

*Corresponding Author

processes remain opaque to the LLM, which receives no explicit structural reasoning chain to explain why those particular chunks were selected, making the LLM struggle to locate the query-related context within those chunks, leading to suboptimal query answering performance on certain tasks [31], [34], [35].

3) **Inadequate Answer:** Although we can simply combine the existing RAG models with the current shortest path-based graph algorithm to provide explicit reasoning chains to aid LLM query answering, improve the model's reasoning ability. However, on tasks that require summarization of current knowledge [31], [34], [35], this simple approach can lead to suboptimal performance, as the shortest path on the graph can be overly simple, and can omit many query-related facts, leading to incomplete reasoning chains, which makes the model tend to generate inadequate query answer with low information coverage and faithfulness (See Table VI and Section IV-D).

To address these issues, we propose a novel **A**dvanced **G**raph-based **R**etrieval-**A**ugmented **G**eneration framework, called AGRAG. We divide the AGRAG framework into three steps: In Step 1 Data Preparation, AGRAG first extracts entities with a TFIDF [36] based entity extraction method, avoiding LLM hallucination, and resulting in better graph construction accuracy. Then, AGRAG applies LLM to detect the possible relation w.r.t. the extracted entity and the text chunk, formulating entity-relation-entity triples, namely knowledge triples. Then these knowledge triples, as well as the text chunks involved, together formulate a Knowledge Graph (KG). In Step 2 Graph Retrieval, AGRAG maps the query to knowledge triples in KG based on their semantic similarity, then calculate the PPR score [33] as node influence based on the KG structure and the mapped triples. In the meantime, AGRAG also calculates edge weights based on the semantic similarity between the query and the edge's corresponding triple. After that, AGRAG generates a Minimum Cost and Maximum Influence (MCMI) reasoning subgraph via a greedy algorithm, which is calculated based on the shortest spanning tree of the mapped triples. It tries to maximize the node influence score, while minimize the edge cost in the expansion of each node, in order to avoid introducing noise. Compared to other shortest path-based algorithms, the MCMI subgraph allows for more complex structures, such as rings, which results in reasoning chains containing more query-related entities and relations, making them more comprehensive. This, in turn, helps the model perform better on complex tasks that require summarization, resulting in better reasoning ability and more comprehensive query answers. In Step 3 Hybrid Retrieval, AGRAG carries out hybrid retrieval based on the average of dense representation vector similarity score and sparse BM25 [37] keyword matching score between query and text chunks, providing extra contexts for query answering. At last, AGRAG will input the resulting subgraph from the Graph Retrieval step and chunks from hybrid retrieval to LLM for final query answering.

We summarize our novel contributions as follows:

- We present the AGRAG framework, which consists of three steps: Data Preparation, Graph Retrieval, and Hybrid Retrieval, aiming to address the Inaccurate Graph Construction, Poor Reasoning Ability, and Inadequate Answer issues of current Graph-based RAG models.
- We design a statistic-based entity extraction method, avoiding the hallucination caused by LLM entity extraction, and improving the pipeline effectiveness.
- We propose the MCMI subgraph generation problem, prove it to be NP-hard, and design a greedy algorithm, ensuring AGRAG's generalizability to complex tasks.
- According to our experiments, AGRAG achieves better effectiveness and efficiency compared with current state-of-the-art RAG models on six tested tasks.

II. Related Works

In this section, we first introduce the traditional RAG models in Section II-A and then discuss the more advanced Graph-based RAG models in Section II-B. The important notations used in this paper are listed in Table I.

A. Traditional RAG Models

Recently, Large Language Model (LLM)-based models have undergone rapid development [38]–[46] and have been successfully adapted to various database related tasks, including data discovery [47]–[49], entity or schema matching [50]–[53], and natural language to SQL conversion [54]–[57].

Although LLMs are inherently capable of inference without fine-tuning [1]–[5], the lack of fine-tuning with up-to-date and domain specific knowledge can lead LLMs to generate incorrect answers. As they lack task-specific knowledge, these incorrect answers are often referred to as hallucinations [58]. To mitigate hallucinations, researchers try to directly retrieve text documents outside of LLM based on the query as side information to help the LLM query answering [59]–[68], lead to the emergence of early RAG models, namely traditional RAG models. The earliest LLM-RAG model is the simple NaiveRAG [15], when indexing, it splits the texts into chunks, and encodes them to semantic vector representations. When retrieval, the model retrieves several text chunks with top semantic similarity to the query.

Based on NaiveRAG, researchers have proposed various walking-based models to further tailor the text chunks and improve the retrieval quality. For example, Propositionizer [59] applies a fine-tuned LLM to convert the text chunks into atomic expressions, namely propositions, to facilitate fine-grained information retrieval. Prompt compressor models [69], [70], such as LongLLMLingua [69] apply LLM's generation perplexity to filter out unimportant tokens in the text chunks, try to maintain a more concise retrieval result for LLM input. However, the retrieval procedures of these models remain unstructured and struggle to capture the nuanced semantic and structured relationship within the text chunks, which impedes the effectiveness of LLMs [15]–[18].

B. Graph-based RAG Models

To overcome the aforementioned limitations of traditional RAG models, researchers have begun to develop Graph-based RAG models [23]–[29]. These models usually extract graph components with LLMs from text corpora to index them into graphs. When retrieval, these models will utilize the graph topology structure to capture complex relationships between entities within and across text chunks.

1) ***Tree-based Models****:* To address the limitation of flat retrieval in capturing long-range dependencies and thematic information, tree-based models organize text chunks into hierarchical structures. RAPTOR [60] recursively clusters and summarizes text chunks to construct a tree from the bottom up, enabling the retrieval of information at varying levels of abstraction to answer high-level thematic queries. Building on this, SiReRAG [71] constructs dual index trees to capture both semantic similarity and entity-based relatedness. It builds a similarity tree via recursive summarization and a relatedness tree that groups fine-grained propositions sharing the same entities, thereby indexing both similar and related information to support complex multihop reasoning.

However, these tree-based models can only generate tree-structured reasoning chains, which restricts their adaptivity to different tasks.

2) ***Community-based Models****:* Early Graph-based RAG models typically retrieve specific neighborhoods from the graph based on the input query. The pioneering work in this category is GraphRAG [23], it aggregates nodes into communities and generates community summaries to encapsulate global information. To reduce the computational overhead of community detection, LightRAG [24] retrieves semantically similar entities along with their 1-hop graph neighborhoods and corresponding text passages, eliminating the need for pre-computed summaries.

Moving beyond fixed neighborhoods, G-Retriever [72] formulates subgraph retrieval as a Prize-Collecting Steiner Tree (PCST) optimization problem, identifying connected subgraphs that maximize relevance while minimizing costs. Recently, MIXRAG [73] introduces a Mixture-of-Experts (MoE) framework to handle diverse query intents, dynamically routing queries to specialized entity, relation, or subgraph experts and employing a query-aware graph encoder to filter noise.

Moreover, all these community-based models can be limited in capturing information beyond a small local neighborhood, thereby restricting their long-hop reasoning ability [74], [75].

3) ***Walking-based Models****:* To further enhance the long-range graph reasoning capabilities of community-based models, researchers have proposed walking-based models [25], [27], [29], [32], which apply walking-based graph algorithms, to detect and capture global topological information within the Knowledge Graphs (KGs).

Among these, HippoRAG [27] and Fast-GraphRAG [32] utilize the Personalized PageRank (PPR) algorithm to traverse the constructed KG and assign relevance scores to graph nodes with respect to the input query. During retrieval, these models select text chunks based on the aggregated PPR scores of the graph nodes they contain. StructRAG [25] trains a router module that learns to identify the optimal reasoning path over the graph based on the input query. HippoRAG2 [29] further improves upon HippoRAG by incorporating text chunks as passage nodes directly into the KG. It retrieves text chunks based on the individual PPR scores of their corresponding passage nodes within the graph.

In this paper, we propose a novel graph-based and efficient RAG framework, termed AGRAG. AGRAG first constructs a KG using a statistics-based entity extraction method, and then assigns graph's node weights based on both the KG structure and the input query. Finally, it generates a Minimum Cost and Maximum Influence (MCMI) reasoning subgraph to provide the LLM with explicit reasoning chains that maximize query-relevant information while minimizing noise during retrieval.

Table I: Summary of important notations.

Notations	Meanings
t	The text chunk.
q	The input query.
$\mathcal{T}$, $\mathcal{T}_c$	Original texts and split text chunks.
l_t, l_o	Text chunk and token overlap length.
s, c	The node influence score and edge cost.
$\mathcal{S}_\mathcal{V}$, $\mathcal{C}_\mathcal{E}$	Set of node influence score and edge cost.
τ, ϕ	Entity extraction and synonym threshold.
b	The maximum n-gram in extity extraction.
k_a, k_r	Number of raw mapped triples, and hybrid retrieved chunks.
f	The mapped triplet fact in KG.
$\mathcal{F}_{\mathcal{KG}}$	All triplet facts within the KG.
$\mathcal{F}_{raw}$, $\mathcal{F}$	Raw and mapped triplet facts after filter.
v	A node (entity) in KG.
$e_{(u,v)}$	An edge (relation) between node u and v.
$\mathcal{KG}(\mathcal{V},\mathcal{E})$	KG with entity set $\mathcal{V}$ and relation set $\mathcal{E}$.
d, $\mathbf{p}$, $\mathbf{P}$	The damping factor, personalization vector and transition probability matrix of PPR.
$\mathcal{N}(\cdot)$	1-hop neighbor set of node or subgraph in KG.
r_{MCMI}	MCMI subgraph's total cost-score ratio.
g	MCMI subgraph's text-formed graph string.
p_{re}, p_{tf}, p_{gen}	LLM prompt for relation extraction, triplet filtering, and answer generation.
$AS(\cdot)$, $HS(\cdot)$	The anchor score and hybrid retrieval score.
$EC(\cdot)$	The edge cost function.

III. Methodology

In this section, we first introduce the overall AGRAG framework, then detail its three core steps: Data Preparation, Graph Retrieval, and Hybrid Retrieval.

A. Framework Overview

As shown in Figure 1, AGRAG consists of three steps, i.e., Data Preparation, Graph Retrieval, and Hybrid Retrieval.

Step 1: Data Preparation. In this step, AGRAG constructs a Knowledge Graph (KG) $\mathcal{KG}(\mathcal{V},\mathcal{E})$ based on the given text corpus $\mathcal{T}$. Here, we substitute other Graph-based RAG baselines' LLM entity extraction to a statistics-based method based on TFIDF [36], and extract entities from text chunks in $\mathcal{T}_c$ that split from the texts in corpus $\mathcal{T}$. This method can avoid LLM hallucination, and hence reduce the noise in the constructed graph, leading to better effectiveness. After entity

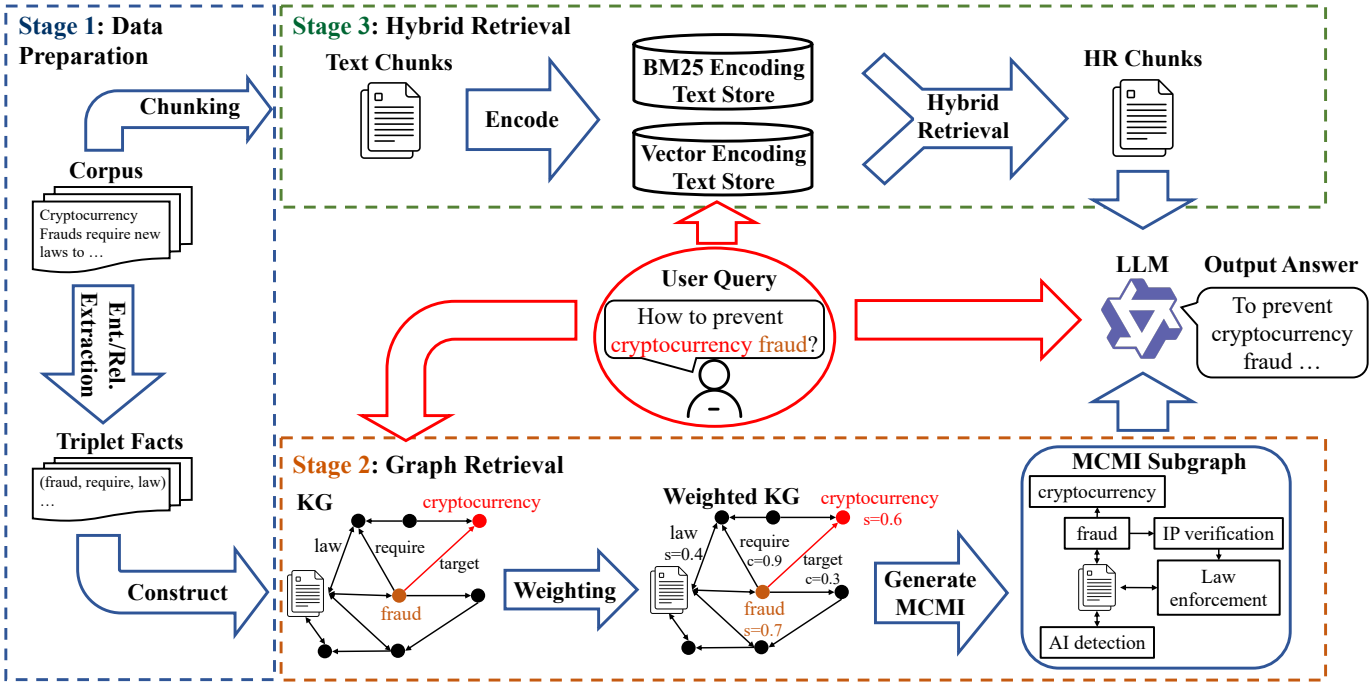


Figure 1: An overview of AGRAG. In Stage 1, AGRAG constructs a KG based on the text corpus. In Stage 2, the query is mapped to relevant triplet facts in KG, then node score and edge weights are assigned based on PPR algorithm and the semantic similarity between query and each fact, respectively. In Stage 3, AGRAG generates an MCMI subgraph, providing explicit reasoning chains in addition to the most semantically similar text chunks by Hybrid Retrieval (HR) to support LLM query answering.

extraction, we apply LLM to detect relation edges w.r.t. the extracted entity nodes and the text chunk. Furthermore, we also construct text chunks into KG as passage nodes, and link them to all their extracted entity nodes with node-to-passage edges. The detailed pseudo code of Step 1 is shown in Algorithm 1, and more details can be found in Section III-B.

Step 2: Graph Retrieval. In this step, we firstly map the query q to KG fact triples considering their semantic similarities, after that, we calculate the PPR score [33] based on these mapped fact triples F as the node influence score $\mathbf{s}$. On the other hand, we calculate the edge costs $\mathcal{C}_{\mathcal{E}}$ considering the semantic similarity between query and facts' vector representations. the higher their similarity is, the lower the cost is. Compared with solely applying PPR to score nodes for retrieval, the edge cost further provides constraints on the involved nodes based on the semantic correspondence between query and facts, removing the need for the human-defined PPR retrieval threshold. After that, we initialize the MCMI subgraph with the generated MCST, and apply our greedy algorithm by iteratively expanding the MCMI subgraph by selecting its neighboring node with the lowest edge cost-influence score ratio (i.e., c/s). If the lowest ratio is larger than the total ratio of the current MCMI, the iteration stops and the current MCMI subgraph is returned. We prove the MCMI generation problem to be NP-hard, and our greedy algorithm in Algorithm 3 can achieve a 2-approximation. Please refer to Section III-C for further details.

Step 3: Hybrid Retrieval. In this step, we perform hybrid retrieval and obtain text chunks based on their BM25 sparse similarity score, and dense vector representation similarity score with the query. These chunks will be input together with the MCMI subgraph to the LLM for the final query answering, providing extra contexts.

B. Step 1: Data Preparation

In this subsection, we utilize a TFIDF [36] based entity extraction approach to substitute the LLM-based approach of other Graph-based RAG models, reduce LLM hallucination, resulting in less noise generated from the early entity extraction step, and hence less error propagation, address the inaccurate graph construction issue, leading to better effectiveness. The pseudo code of AGRAG's graph construction procedure is shown in Algorithm 1.

To begin with, we split texts in $\mathcal{T}$ into fixed-length chunks with a certain amount of token overlap between chunks:

$$\mathcal{T}_c = \text{Split}(\mathcal{T}, l_t, l_o), \tag{1}$$

where l_t and l_o denotes the length of text chunks and token overlap between chunks. Next, we apply the statistics-based entity extraction approach on the text chunks $\mathcal{T}_c$. Specifically, our approach modifies the TFIDF score [36] as the entity extraction score $\text{ER}(v,t)$ for each term v in text chunk t as an entity, to make the value of ER fall in $(0,1)$. Here, our entity detection is based on an n-gram setting with the maximum n-gram (number of words in an entity) as b, and the equation of $\text{ER}(v,t)$ is as follows:

$$\text{ER}(v,t) = \frac{\text{count}(v,t)}{|t| log(|\mathcal{T}_c|+1)} \times log \frac{|\mathcal{T}_c|+1}{|t_j : v \in t_j,\ t_j \in \mathcal{T}_c|+1}, \tag{2}$$

Algorithm 1: Indexing Algorithm of AGRAG

Input : $\mathcal{T}$: The text corpus for graph construction.
Output: $\mathcal{KG}(\mathcal{V}, \mathcal{E})$: The KG constructed.

Let $\mathcal{V} \leftarrow \emptyset$, $\mathcal{E} \leftarrow \emptyset$;
Get text chunk set $\mathcal{T}_c$ from $\mathcal{T}$: $\mathcal{T}_c \leftarrow \text{Chunking}(\mathcal{T})$;
for each text chunk $t \in \mathcal{T}_c$ **do**
  Extract entity set $\mathcal{V}_t$ based on Equation (3);
  Carry out relation extraction with LLM:
   $\mathcal{E}_t \leftarrow \text{LLM}(\mathcal{V}_t, t, p_{\text{re}})$;
  $\mathcal{V} \leftarrow \mathcal{V} \cup \mathcal{V}_t$, $\mathcal{E} \leftarrow \mathcal{E} \cup \mathcal{E}_t$;
Add $\mathcal{T}_c$ to $\mathcal{V}$ as passage nodes: $\mathcal{V} \leftarrow \mathcal{V} \cup \mathcal{T}_c$;
for each entity node $v \in \mathcal{V}$ **do**
  for each text chunk $t \in \mathcal{T}_c$ **do**
    if $v \in t$ **then**
      Add passage edge between v and t:
       $\mathcal{E} \leftarrow \mathcal{E} \cup \{e_{(v,t)}\}$;
return $\mathcal{KG}(\mathcal{V}, \mathcal{E})$;

where $\mathcal{T}_c$ denotes all text chunks obtained from Equation (1), $\text{count}(v, t)$ is the number of times that the term v appears in the text t, and $|t_j : v \in t_j,\ t_j \in \mathcal{T}_c|$ denotes the number of texts in the corpus $\mathcal{T}_c$ that contain the keyword v. Here, our entity extraction procedure will extract words or phrases with $\text{ER}(v, t) > \tau$ as the entity nodes $\mathcal{V}_t$ for text chunk t:

$$\mathcal{V}_t = \{v \,|\, \text{ER}(v, t) > \tau\}. \tag{3}$$

Entities in $\mathcal{V}_t$ are later served as graph nodes, hence we denote them as entity nodes. For our entity extraction procedure, the entity boundary b and entity extraction threshold τ are considered as hyper-parameters, we further analyze their setting in Section IV-E.

After entity extraction, we extract relations from text chunks $t \in \mathcal{T}_c$ based on its extracted entities $\mathcal{V}_t$. Align with other Graph-based RAG models, we also apply LLM at this step:

$$\mathcal{E}_t = \text{LLM}(\mathcal{V}_t,\ t,\ p_{\text{re}}), \tag{4}$$

where $\mathcal{V}_t$ denotes the extracted entities of text chunk t, and in the relation extraction prompt p_{re}, we ask the LLM to detect relations between entities in $\mathcal{V}_t$ w.r.t. text chunk t. Although we also apply LLM for relation extraction, we input less noise in this step, mitigating the error propagation, as our statistics-based entity extraction avoids LLM hallucination, hence resulting in a more comprehensive and less noisy $\mathcal{V}_t$ as the Equation (4) input.

After relation extraction, we add text chunks in $\mathcal{T}_c$ to $\mathcal{V}$ as passage nodes, and add passage edges $e_{(v,t)}$ between entities contained within the text chunks to $\mathcal{E}$, the respective relation of these edges is namely *Contains*. By introducing passage nodes, we integrate contextual information into the constructed KG, and the entity-passage edges can capture the inter-text correlations, make our model able to generate reasoning paths across different text chunks [29].

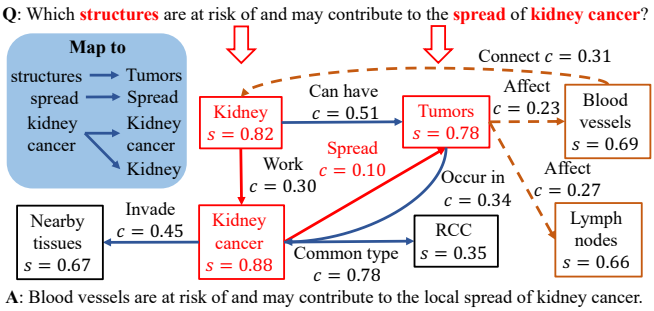


Figure 2: An example of MCMI. Red triples indicate those mapped from the query. Our algorithm first constructs the MCMI based on a Steiner tree rooted at these mapped triples, then iteratively expands it by adding neighboring triples (in orange) with the lowest cost-score ratio, continuing until no neighboring triple has its ratio less than the current MCMI's ratio.

Furthermore, we add synonym edges $\mathcal{E}_{\text{syn}}$ between entity nodes in $\mathcal{V}$ that have similarities larger than the synonym threshold ϕ, the relation of these edges is namely *synonym*:

$$\mathcal{E}_{\text{syn}} = \left\{e_{(u,v)} \,\middle|\, \text{cosine_sim}(\mathbf{u}, \mathbf{v}) > \phi\right\}, \tag{5}$$

At last, we combine edge set $\mathcal{E}_t$ for all text chunks t and passage edges together, as well as the synonym edges $\mathcal{E}_{\text{syn}}$, to form the total edge (relation) set of the KG:

$$\mathcal{E} = \bigcup_{t \in \mathcal{T}_c} \mathcal{E}_t \ \cup \bigcup_{v \in \mathcal{V}_t,\ t \in \mathcal{T}_c} e_{(v,t)} \ \cup \mathcal{E}_{\text{syn}}, \tag{6}$$

here, if the entity v does not exist in text chunk t, $e_{(v,t)} = \emptyset$.

C. Step 2: Graph Retrieval

In this subsection, we introduce the graph retrieval procedure of AGRAG. Given the KG $\mathcal{KG}(\mathcal{V}, \mathcal{E})$ constructed at the Step 1, we first map the query q to $\mathcal{KG}$ and obtain raw mapped triplet facts $\mathcal{F}_{\text{raw}}$. To achieve this, we encode each triplet fact in KG $f \in \mathcal{F}_{\mathcal{KG}}$ and the query q into vector representations $\mathbf{q}$ and $\mathbf{f}$ with encoder $\mathcal{M}$: $\mathbf{q} \leftarrow \mathcal{M}(q)$, then select facts with top-k dense semantic mapping score MS to query q based on their cosine similarity:

$$\text{MS}(\mathbf{q}, \mathbf{f}) = \text{cosine_sim}(\mathbf{q}, \mathbf{f}), \tag{7}$$

$$\mathcal{F}_{\text{raw}} = \{f \in \mathcal{F}_{\mathcal{KG}} \,|\, f \in \arg\top_{k_a} (MS(\mathbf{q}, \mathbf{f}))\}, \tag{8}$$

where k_a is a hyperparameter that controls the number of raw mapped triples extracted, and $\mathbf{f}$ are obtained through encoding the text concatenation of entities and relation of the triplet fact.

Further, as the triplets in KG are extracted in a query unaware manner, query irrelevant triplets can exist in the raw mapped triplet facts $\mathcal{F}_{\text{raw}}$, hence we apply LLM-based filter on $\mathcal{F}_{\text{raw}}$ to generate the mapped triplet facts $\mathcal{F}$, to reduce the query irrelevant noise:

$$\mathcal{F} = \text{LLM}(\mathcal{F}_{\text{raw}}, q, p_{\text{tf}}), \tag{9}$$

where p_{tf} denotes the LLM prompt for triple filtering.

Next, based on the mapped triplet facts $\mathcal{F}$ and the KG structure, we calculate the Personalized PageRank (PPR) [33] score as our node influence score $\mathbf{s}$ for node set $\mathcal{V}$:

$$\mathbf{s} = \lim_{i \to \infty} \text{PPR}^i,\ \text{PPR}^i = (1 - d) \times \mathbf{p} + d \times \mathbf{P} \cdot \text{PPR}^{i-1}, \tag{10}$$

Algorithm 2: Graph Weighting Algorithm

Input : $\mathcal{KG}(\mathcal{V}, \mathcal{E})$: The indexed KG;
$\mathcal{F}$: A set of query-mapped triplet facts in $\mathcal{KG}$;
$\mathbf{q}$: The vector representation of the query q.
Output: $\mathcal{KG}(\mathcal{V}, \mathcal{E}, \mathcal{S_V}, \mathcal{C_E})$: The weighted KG.

Assign node weights $\mathcal{S_V}$ with PPR score w.r.t. $\mathcal{F}$ based on Equation (10): $\mathbf{s} \leftarrow \lim_{i\to\infty} \text{PPR}^i$;
for $e_{(u,v)} \in \mathcal{E}$ **do**
  Assign edge costs $c_{e_{(u,v)}}$ based on Equation (14): $c_{e_{(u,v)}} \leftarrow (1 - \text{MS}(\mathbf{q}, \mathbf{f}_{e_{(u,v)}}))/2$;
  $\mathcal{C_E} \leftarrow \mathcal{C_E} \cup \{c_{e_{(u,v)}}\}$;
return $\mathcal{KG}(\mathcal{V}, \mathcal{E}, \mathcal{S_V}, \mathcal{C_E})$;

and we have the node score set $\mathcal{S_V} = \{\mathbf{s}[v] | v \in \mathcal{V}\}$. To align with [29], we set the damping factor d to 0.5, and converge the PPR calculation when it changes less than 10^{-7}; $\mathbf{p} \in \mathbb{R}^{|\mathcal{V}|}$ denotes the personalization vector that depends on the mapped triples $\mathcal{F}$:

$$\mathbf{p}[u] = \begin{cases} 1, & \text{if } u \in \mathcal{F}, \\ 0, & \text{otherwise.} \end{cases} \tag{11}$$

The $\mathbf{P} \in \mathbb{R}^{|\mathcal{V}\times\mathcal{V}|}$ in Equation (10) denotes the transition probability matrix, and can be obtained by:

$$\mathbf{P}[u][v] = \begin{cases} \frac{1}{|\mathcal{N}(u)|}, & \text{if } e_{(u,v)} \in \mathcal{E}, \\ \frac{1}{|\mathcal{V}|}, & \text{otherwise,} \end{cases} \tag{12}$$

where $\mathcal{N}(u)$ denotes the 1-hop neighbor set of node u in the KG. To ensure the PPR's faster convergence and more stable iterative computation [76], as well as normalizing the score distribution [77]. Following [29], we set the transition probability for non-edges to $\frac{1}{|\mathcal{V}|}$, and after calculating the PPR score, we multiply the PPR score of all passage nodes by a balance factor 0.05, to avoid over concentration on them. After that, we can obtain the node influence score s_v for each node $v \in \mathcal{V}$ as follows:

$$s_v = \mathbf{s}[v], \quad \forall v \in \mathcal{V}. \tag{13}$$

At last, we calculate the edge cost $\mathcal{C_E}$ based on the similarity between query q's representation $\mathbf{q}$ and relation edge $e_{(u,v)}$'s respective triplet facts $f_{e_{(u,v)}}$'s representation $\mathbf{f}_{e_{(u,v)}}$:

$$\mathcal{C_E} = \{c_{e_{(u,v)}} \mid e_{(u,v)} \in \mathcal{E}\}, \quad c_{e_{(u,v)}} = \frac{1 - \text{MS}(\mathbf{q}, \mathbf{f}_{e_{(u,v)}})}{2}. \tag{14}$$

Moreover, to guarantee the KG's connectivity, we connect all passage nodes to a pseudo node, with the pseudo edge (relation) namely *pseudo_relation*, and the cost of these pseudo edges is set to 10 to penalize paths through the pseudo node, aligning with the setting of HippoRAG2 [29].

Next, we introduce the Minimum Cost Maximum Influence (MCMI) subgraph generation problem, which can provide more comprehensive reasoning chains and result in more comprehensive answers in tasks that require knowledge summarization. According to Figure 2, the MCMI subgraph can include cyclic graph structures to cover key query related triples (colored in orange), which are not covered by simple node matching or Steiner tree, hence address the Poor Reasoning Ability and Inadequate Answer issue of current Graph-based RAG models. Here, we give a formal definition of the MCMI subgraph generation problem:

Definition 1 (Minimum Cost Maximum Influence Subgraph Generation Problem)**.** *Given a node-weighted and edge-weighted, connected graph $\mathcal{G}(\mathcal{V}, \mathcal{E}, \mathcal{S_V}, \mathcal{C_E})$ and a set of mapped triples $\mathcal{F}$, let $\mathcal{V}_{term}$ be the set of nodes in $\mathcal{F}$, called terminals. Let $r \in \mathcal{V}_{term}$ be a designated root terminal. The goal is to generate a subgraph $\mathcal{G}_{MCMI}(\mathcal{V}', \mathcal{E}', \mathcal{S}'_{\mathcal{V}}, \mathcal{C}'_{\mathcal{E}})$ that contains all terminals $\mathcal{V}_{term}$ such that every terminal $v \in \mathcal{V}_{term}$ is reachable from root terminal r, while minimizing the total cost-score ratio $r(\mathcal{G}_{MCMI})$:*

$$r(\mathcal{G}_{MCMI}) = \sum_{e_{(u,v)} \in \mathcal{E}'} \frac{c_{e_{(u,v)}}}{s_u + s_v}, \tag{15}$$

where $s_v = \mathbf{s}[v]$ denotes the Personalized PageRank (PPR) score of node v, as defined in Equation (10)*, and $c_{e_{(u,v)}}$ represents the edge cost of $e_{(u,v)}$, as defined in Equation* (14)*.*

By minimizing the cost-score ratio in Equation (15), we can ensure the generated subgraph maximizes the inclusion of high-influence, query-relevant nodes while strictly penalizing low-similarity connections to effectively filter out noise.

Theorem 1. *The MCMI subgraph generation is NP-hard.*

Proof Sketch. We establish NP-hardness by constructing a bijection between the MCMI problem and the classic Steiner Tree problem via a transformed weight function. Due to the space limitation, we put our proof at https://github.com/Wyb0627/AGRAG/blob/main/proof.pdf. □

Since the MCMI Subgraph Generation problem is NP-hard, it is infeasible to obtain the optimal result in polynomial time. Therefore, we propose a greedy algorithm (Algorithm 3) based on the Mehlhorn's algorithm [78] to solve this problem.

To begin with, we initialize the MCMI as the Minimum Cost Steiner Tree (MCST) through all mapped triplet facts $\mathcal{F}$ in the weighted graph $\mathcal{G}(\mathcal{V}, \mathcal{E}, \mathcal{S_V}, \mathcal{C_E})$ (line 1 to line 10 of Algorithm 3), where score $s_v \in \mathcal{S_V}$ of each node v, and cost $c_{e_{(u,v)}} \in \mathcal{C_E}$ of each edge e are obtained with Equation (13) and Equation (14), respectively:

$$\mathcal{G}_{\text{MCMI}}(\mathcal{V}_{\text{MCMI}}, \mathcal{E}_{\text{MCMI}}) = \text{MCST}(\mathcal{F}, \mathcal{G}), \tag{16}$$

where $\text{MCST}(\mathcal{F}, \mathcal{G})$ denotes the Minimum Cost Steiner tree generation algorithm.

Next, we calculate the ratio of the edges' cost and their connected nodes' influence score (i.e., c/s), and check this ratio of all nodes in the current MCMI's 1-hop neighborhood $\mathcal{N}(\mathcal{G}_{\text{MCMI}})$, if this ratio is less than that of the current MCMI subgraph's total cost-score ratio, we add this node (entity) and the corresponding edge (relation) to the MCMI, recalculate the

MCMI's cost-score ratio and continue the iteration, until all neighbor node's cost-score ratio is more than or equal to the MCMI subgraph's total cost-score ratio. That is, for each edge $e_{(u,v)}$, we have (line 10 to line 16 of Algorithm 3):

$$\mathcal{V}_{\text{MCMI}} = \begin{cases} \mathcal{V}_{\text{MCMI}} \cup v, & \text{if } \frac{c_{e_{(\text{MCMI},v)}}}{s_v} < r(\mathcal{G}_{\text{MCMI}}), \\ \mathcal{V}_{\text{MCMI}}, & \text{otherwise}, \end{cases}$$
$$\mathcal{E}_{\text{MCMI}} = \begin{cases} \mathcal{E}_{\text{MCMI}} \cup e_{(\text{MCMI},v)}, & \text{if } \frac{c_{e_{(\text{MCMI},v)}}}{s_v} < r(\mathcal{G}_{\text{MCMI}}), \\ \mathcal{E}_{\text{MCMI}}, & \text{otherwise}, \end{cases} \tag{17}$$

where $\mathcal{V}_{\text{MCMI}}$ and $\mathcal{E}_{\text{MCMI}}$ denote the node and edge set of current MCMI subgraph; $e_{(\text{MCMI},v)}$ denotes the outgoing edges from the MCMI subgraph to its 1-hop neighborhood.

After the MCMI subgraph generation, we serialize the generated MCMI subgraph into a text graph string g:

$$g = \text{StrConcat}(\mathcal{V}_{\text{MCMI}}, \mathcal{E}_{\text{MCMI}}). \tag{18}$$

Running Example: As shown in Figure 2, if we simply calculate the minimum cost Steiner tree within the given KG, the algorithm will simply return the two mapped fact (*Kidney, Work, Kidney cancer*), and (*Kidney, Spread, Tumors*) as the reasoning path, and the LLM's response answer would likely be *Tumors*. However, if we go one step further, then within the generated cyclic graph, we can have more complete reasoning information regarding the query, and involve the correct answer *Blood vessels*.

We further prove our MCMI generation algorithm's MCST generation step (Lines 1-10 in Algorithm 3) can still have a 2-approximation rate in Theorem 2. And its MCMI generation step (Lines 11-15 in Algorithm 3) strictly monotonically gains information and terminates within at most $|\Omega|$ iterations in Theorem 3, where Ω denotes the candidate edge set of MCMI. Together, they establish the effectiveness and efficiency of our MCMI generation Algorithm 3.

Theorem 2. *The MCST generation step (Lines 1-10 in Algorithm 3) guarantees a 2-approximation.*

Proof Sketch. We bound the cost of the generated subgraph by comparing the Minimum Spanning Tree (MST) on the auxiliary graph H to a shortcutted Euler tour (by depth-first traversal) of the optimal Steiner tree. Due to space limitation, we put our proof at https://github.com/Wyb0627/AGRAG/blob/main/proof.pdf. □

Theorem 3. *The greedy expansion of the MCMI generation step (Lines 11-15 in Algorithm 3) strictly monotonically gains information and terminates within at most $|\Omega|$ iterations, where Ω denotes the candidate edge set of MCMI:*

$$\Omega = \left\{ e_{(u,v)} \in \mathcal{E} \setminus \mathcal{E}_{MCST} \mid \frac{c_{e_{(u,v)}}}{s_v} < r(\mathcal{G}_{MCST}) \right\} \tag{19}$$

Proof Sketch. We prove the MCMI generation steps' strict monotonic improvement by showing the algorithm only adds edges if their marginal cost-score ratio is strictly lower than the current subgraph's ratio. Termination is guaranteed because the expansion process is restricted to a finite set of candidate edges. Due to the space limitation, we put our proof at https://github.com/Wyb0627/AGRAG/blob/main/proof.pdf. □

Algorithm 3: MCMI Subgraph Generation Algorithm

Input : $\mathcal{KG}(\mathcal{V}, \mathcal{E}, \mathcal{S}_{\mathcal{V}}, \mathcal{C}_{\mathcal{E}})$: The weighted KG;
$\mathcal{F}$: A set of query-mapped triplet facts in $\mathcal{KG}$.
Output: $\mathcal{G}_{\text{MCMI}}(\mathcal{V}_{\text{MCMI}}, \mathcal{E}_{\text{MCMI}})$: The MCMI subgraph.

1 Compute an approximated minimum cost Steiner tree (MCST) $\mathcal{G}$ of $\mathcal{KG}$;
2 Let $\mathcal{V}_{\text{term}} \leftarrow \mathcal{F} \cap \mathcal{V}_{\text{MCST}}$ be the terminal nodes in MCST;
3 Construct weighted auxiliary graph H: $\mathcal{V}_H \leftarrow \mathcal{V}_{\text{term}}$;
4 **for** each pair of terminal nodes $u, v \in \mathcal{V}_{\text{term}}$ **do**
5 Find the shortest path $P_{(u,v)}$ in $\mathcal{G}$ from u to v;
6 Let $c^H_{(u,v)} \leftarrow \sum_{e \in P_{(u,v)}} c_e$, where $c_e \in \mathcal{C}_{\mathcal{E}}$;
7 Compute an approximated MCST $\mathcal{G}'$ of H;
8 Let $\mathcal{V}_{\text{MCMI}} \leftarrow \emptyset$, $\mathcal{E}_{\text{MCMI}} \leftarrow \emptyset$;
9 **for** each edge $e_{(u,v)} \in \mathcal{G}'$ **do**
10 Add nodes and edges from shortest path $P_{(u,v)}$ in MCST to $\mathcal{V}_{\text{MCMI}}$ and $\mathcal{E}_{\text{MCMI}}$;
11 Calculate $r(\mathcal{G}_{\text{MCMI}})$ with Equation (15);
12 **while** the node v in MCMI's 1-hop neighborhood with lowest $c_{e_{(\text{MCMI},v)}}/s_v$ satisfies $c_{e_{(\text{MCMI},v)}}/s_v < r(\mathcal{G}_{\text{MCMI}})$ **do**
13 Recalculate $r(\mathcal{G}_{\text{MCMI}})$ with Equation (15);
14 $\mathcal{V}_{\text{MCMI}} \leftarrow \mathcal{V}_{\text{MCMI}} \cup \{v\}$;
15 $\mathcal{E}_{\text{MCMI}} \leftarrow \mathcal{E}_{\text{MCMI}} \cup \{e_{(\text{MCMI},v)}\}$;
16 **return** $\mathcal{G}_{\text{MCMI}}(\mathcal{V}_{\text{MCMI}}, \mathcal{E}_{\text{MCMI}})$;

D. Step 3: Hybrid Retrieval

Other than Graph retrieval, we also apply Hybrid Retrieval (HR) to further enrich the context for query answering. In Hybrid Retrieval, we retrieve the most similar text chunks $\mathcal{T}_{HR}$ from all text chunks $t \in \mathcal{T}_c$ to the query's vector representation $\mathbf{q}$, based on the hybrid retrieval score $\text{HS}(\cdot)$:

$$\mathcal{T}_{\text{HR}} = \{ t \in \mathcal{T}_c \mid t \in \arg \top_{k_r} (\text{HS}(q,t)) \}, \tag{20}$$
$$\text{HS}(q,t) = \frac{\text{MS}(\mathbf{q},\mathbf{t}) + \text{BM25}(q,t)}{2} \tag{21}$$

where k_r denotes the hyperparameter that controls the number of hybrid retrieved text chunks, $\mathbf{t}$, $\mathbf{q}$ denotes the vector representation of text chunk t and query q, respectively; $\text{HS}(q,t)$, $\text{BM25}(q,t)$ and $\text{MS}(\mathbf{q},\mathbf{t})$ denotes the hybrid retrieval score, the BM25 [37] sparse retrieval score, and the dense representation vectors' cosine similarity score between query q and text t in Equation (7).

After obtaining the textual MCMI subgraph string g and the chunks $\mathcal{T}_{\text{HR}}$ from hybrid retrieval, we input them together with the query q and the query answer generation LLM instruction p_{gen}, to ask the LLM to give an answer to query q based on these provided contents.

$$\text{answer} = \text{LLM}(q, g, \mathcal{T}_{\text{HR}}, p_{\text{gen}}), \tag{22}$$

E. Complexity Analysis

*1) **Time Complexity**:* For the entity extraction procedure, the time complexity is $O(|\mathcal{V}||\mathcal{T}_c|)$; Adding passage nodes also requires $O(|\mathcal{V}||\mathcal{T}_c|)$ time. For graph weighting, calculating the PPR score requires $O(|\mathcal{T}_c|(|\mathcal{V}|+|\mathcal{E}|))$ time, and calculate the edge cost requires $O(|\mathcal{E}|)$ time. For AGRAG's Step 3, the time complexity of AGRAG's MCST generation is $O(|\mathcal{E}|+|\mathcal{V}|log|\mathcal{V}|)$ using the Mehlhorn algorithm [78]; The expansion from MCST to MCMI would cost $O(|\mathcal{V}_{\text{MCMI}}|\cdot|\mathcal{N}(\mathcal{V}_{\text{MCMI}})|)$ Hence, the time complexity of AGRAG's MCMI generation at the worst case when $\mathcal{V}_{\text{MCMI}}=\mathcal{V}$ and the graph is fully connected is: $O(|\mathcal{V}|^2+|\mathcal{E}|)$.

Hence, the end to end time complexity of AGRAG is: $O(|\mathcal{T}_c|(|\mathcal{V}|+|\mathcal{E}|)+|\mathcal{V}|^2)$. Since the number of text chunk $|\mathcal{T}_c|$ is less or equal to the number of nodes $|\mathcal{V}|$ in graph, in the worst case, the end to end time complexity of AGRAG is:

$$O(|\mathcal{V}|\cdot|\mathcal{E}|+|\mathcal{V}|^2).$$

*2) **Space Complexity**:* The graph of AGRAG's Step 1 and Step 2 is stored with the dense weighted adjacency matrix, which needs $O(|\mathcal{E}|)$ space; Storing the text chunk would need $O(|\mathcal{T}_c|)$ space; Storing the entity nodes require $O(|\mathcal{V}|)$ space. Hence the total space complexity of AGRAG's Graph Indexing step is: $O(|\mathcal{V}|+|\mathcal{E}|+|\mathcal{T}_c|)$. In AGRAG's Step 3, we only need $O(|\mathcal{E}|)$ to store the graph.

Hence, the total space complexity of AGRAG is:

$$O(|\mathcal{V}|+|\mathcal{E}|+|\mathcal{T}_c|).$$

IV. Experiments

In this section, we first outline the experimental setup in Section IV-A, including details on datasets, and evaluation metrics. Next, we report the experimental results, focusing on both effectiveness and efficiency evaluations, in Section IV-B and Section IV-C, respectively. Finally, we conduct ablation study in Section IV-D, and parameter sensitivity study in Section IV-E.

A. Experiment Settings

*1) **Datasets & Tasks**:* We select the Novel and Medical datasets from GraphRAG-bench [34] to evaluate AGRAG's performance on 4 tasks that have different complexity levels:

- **Fact Retrieval**: The simplest (level 1) task of GraphRAG-bench, to solve this task, the model needs to retrieve isolated facts in the graph with minimal reasoning needed. This task mainly tests the models' precise keyword matching ability.
- **Complex Reasoning (Complex Rea.)**: The level 2 task of GraphRAG-bench, to solve this task, the model needs to connect multiple facts on the graph to form multi-hop, long-range reasoning chains.
- **Contextual Summarization (Contextual Sum.)**: The level 3 task of GraphRAG-bench. Here, the model needs to synthesize fragmented information from the graph into comprehensive summaries.
- **Creative Generation (Creative Gen.)**: The most complex (level 4) task of GraphRAG-bench, to solve this task, the model needs to generate hypothetical responses based on but beyond the retrieved knowledge.

Furthermore, we also apply the text classification task, to test the RAG models' ability to summarize the topic of text passages, and classify them into the correct category within a given label sets.

- **Text Classification**: Given a set of texts $\mathcal{T}$ and a set of label $\mathcal{L}$, the classification task aim to assign each text $t\in\mathcal{T}$ a label $l\in\mathcal{L}$ that best suited to describe its semantic meaning. For text classification task, we select the Web-Of-Science (WOS) dataset [79], and the IFS-Rel dataset [80].

We select one text passage per label from the training sets of the WOS and IFS-Rel datasets to construct the corpus for RAG. During retrieval, instead of retrieving the full text chunk, we enable HippoRAG2 and AGRAG to directly retrieve the corresponding label of each text chunk. The retrieved labels are then used to form a candidate label set for the LLM's text prediction. The LLM is subsequently tasked with selecting the most appropriate label from this candidate set. The detailed dataset and task statistics are shown in Table II.

*2) **Evaluation Metrics**:* In this paper, we apply 5 metrics with the RAGAS [81] package to evaluate AGRAG's performance on 5 different tasks:

a. **Accuracy (ACC.)**: For text classification, the proportion of exact matches between LLM and gold labels. For other tasks, sample is scored 0, 0.5, or 1 by the LLM for inaccurate, partially accurate, or exactly accurate responses.
b. **Recall (REC.)**: Quantifies the model's ability to identify all actual positive instances by calculating the ratio of true positives to the sum of true positives and false negatives, we calculate the macro recall score and apply it to the text classification task, where we have a fixed label set.
c. **ROUGE-L (ROG.)** [82]: Evaluates the answer quality by comparing the longest common subsequence (LCS) between the generated answer G and the gold answer Y: ROUGE-L $=\frac{\text{LCS}(G,Y)}{|G|}$. We apply ROUGE-L to the fact retrieval and the complex reasoning tasks, as the answer of these two tasks are usually a fixed entity or relation.
d. **Coverage (COV.)** [34]: Evaluates answer completeness by assessing whether all necessary reference evidences E are included in the LLM's response R, computed as the proportion of covered evidences to the number of required evidences $|E|$:Cov $=\frac{|\{e\in E\mid M(e,R)\}|}{|E|}$, where $M(e,R)$ is a boolean function indicating whether evidence e is covered by generated response R. We apply the coverage metric to contextual summarization and creative generation tasks, as their answers are usually passages that conclude multiple information points.
e. **Faithfulness (FS.)** [34]: Measures the percentage of answer claims A directly supported by retrieved context C, calculated as the ratio of claims aligned with evidence to the number of total claims A: FS $=\frac{|\{a\in A\mid S(a,C)\}|}{|A|}$, where $S(a,C)$ is a boolean function indicating whether a claim is supported by contexts in C. We apply the faith-

Table II: Statistics of the Novel and Medical dataset from GraphRAG-bench [34] across 4 tasks, and statistics of the WOS [79] and IFS-REL [80] dataset for the text classification task. CG, CS, CR, FR and TC denotes the Creative Generation, Contextual Summarization, Complex Reasoning, Fact Retrieval, and Text Classification task, respectively. '-' denotes the respective task is not included by certain dataset.

Dataset	Chunks	KG Entities	KG Relations	AVG MCST Nodes	AVG MCST Edges	AVG MCMI Nodes	AVG MCMI Edges	CG.	CS.	CR.	FR.	TC.	
								Questions				Texts	Labels
Novel	5,140	63,887	163,169	8.7	10.7	19.7	35.9	67	362	610	971	-	-
Medical	1,030	12,426	52,054	8.9	9.9	23.7	59.5	166	289	509	1,098	-	-
WOS	24	475	923	15.5	17.8	23.8	35.8	-	-	-	-	9,396	133
IFS-Rel	11	185	355	11.0	12.8	17.0	26.3	-	-	-	-	2,560	64

Table III: Model effectiveness on 4 datasets for 5 tasks, all experiments were carried out with Qwen2.5-14B-Instruct LLM backbone.

Category	Task	Creative Gen.						Contextual Sum.				Complex Rea.				Fact Retrieval				Text Classification			
	Dataset	Novel			Medical			Novel		Medical		Novel		Medical		Novel		Medical		WOS		IFS-REL	
	Metric	FS.	ACC.	COV.	FS.	ACC.	COV.	COV.	ACC.	COV.	ACC.	ROG.	ACC.	ROG.	ACC.	ROG.	ACC.	ROG.	ACC.	ACC.	REC.	ACC.	REC.
Traditional	NaiveRAG	0.205	0.307	0.384	0.176	0.577	0.554	0.745	0.621	0.677	0.576	0.120	0.447	0.117	0.635	0.117	0.431	0.182	0.604	0.237	0.257	0.351	0.197
RAG	L.LLMLingua	0.213	0.316	0.377	0.225	0.575	0.496	0.687	0.627	0.657	0.583	0.092	0.434	0.080	0.633	0.115	0.420	0.105	0.591	0.274	0.231	0.378	0.220
Tree	RAPTOR	0.365	0.333	0.373	0.314	0.526	0.553	0.760	0.626	0.633	0.583	0.290	0.460	0.185	0.649	0.133	0.465	0.132	0.609	0.409	0.290	0.293	0.212
-based RAG	SiReRAG	0.324	0.328	0.365	0.295	0.544	0.456	0.754	0.617	0.592	0.560	0.307	0.459	0.269	0.646	0.311	0.451	0.349	0.558	0.430	0.294	0.325	0.220
	G-retriever	0.294	0.167	0.200	0.169	0.371	0.468	0.594	0.404	0.302	0.395	0.146	0.441	0.102	0.396	0.199	0.209	0.148	0.326	0.188	0.129	0.311	0.185
Community	MIXRAG	0.200	0.286	0.337	0.255	0.351	0.306	0.695	0.538	0.409	0.464	0.109	0.386	0.156	0.434	0.102	0.366	0.160	0.463	0.244	0.235	0.362	0.203
-based RAG	GraphRAG	0.427	0.310	0.339	0.345	0.475	0.325	0.758	0.620	0.625	0.506	0.246	0.449	0.250	0.606	0.240	0.445	0.217	0.521	0.253	0.201	0.279	0.202
	LightRAG	0.201	0.314	0.329	0.189	0.567	0.549	0.753	0.628	0.625	0.585	0.223	0.448	0.255	0.637	0.253	0.460	0.283	0.607	0.273	0.224	0.265	0.220
Walking	HippoRAG2	0.496	0.295	0.325	0.347	0.563	0.451	0.735	0.586	0.496	0.560	0.314	0.453	0.332	0.622	0.310	0.470	0.343	0.598	0.424	0.270	0.389	0.246
-based RAG	F.GraphRAG	0.286	0.287	0.376	0.217	0.500	0.396	0.706	0.528	0.507	0.587	0.237	0.371	0.262	0.643	0.318	0.439	0.352	0.610	0.283	0.245	0.307	0.221
	AGRAG	**0.513**	**0.339**	**0.386**	**0.356**	**0.598**	**0.589**	**0.778**	**0.643**	**0.690**	**0.604**	**0.319**	**0.467**	**0.343**	**0.665**	**0.324**	**0.490**	**0.357**	**0.628**	**0.438**	**0.307**	**0.403**	**0.251**

fulness metric to the creative generation task to evaluate the quality of the hypothesis in the LLM's response.

*3) **Baselines**:* We compare AGRAG's performance with 6 baselines from 3 technical categories, they are traditional RAG model NaiveRAG [15] and LongLLMLingua [70]; Community-based model GraphRAG [23], and LightRAG [24]; And walking-based model Fast-GraphRAG [32] and HippoRAG2 [29]. We use the Qwen-2.5-14B-Instruct [83] as the LLM backbone unless further illustrated.

Traditional RAG Models These models directly retrieve text chunks based on their semantic similarity to the queries.

- **NaiveRAG** [15]: NaiveRAG acts as a foundational baseline of current RAG models. It encodes text chunks into a vector database and retrieve the Top-k text chunks based on the vector similarities between queries and text chunks.
- **L.LLMLingua** [70]: LongLLMLingua utilizes LLM's generation perplexity to filter out unimportant LLM input tokens. In this paper, we apply it to compress the retrieved text chunks of NaiveRAG w.r.t. the query and task instruction.

Tree-based Models These models organize text chunks into tree structures to capture long-range dependencies and thematic information.

- **RAPTOR** [60] RAPTOR recursively clusters and summarizes text chunks to construct a tree from the bottom up, enabling the retrieval of information at varying levels of abstraction to answer high-level thematic queries.
- **SiReRAG** [71] Based on RAPTOR, SiReRAG builds similarity and relatedness trees to index respective information, thereby supporting complex multihop reasoning.

Community-based Models These models retrieve one or multi-hop neighborhood of certain graph nodes that correspond to the query.

- **G-retriever** [72] It formulates subgraph retrieval as a Prize-Collecting Steiner Tree (PCST) optimization problem, identifying connected subgraphs that maximize relevance while minimizing costs.
- **MIXRAG** [73] It introduces a Mixture-of-Experts (MoE) framework to handle diverse query intents, dynamically routing queries to specialized experts and employing a query-aware graph encoder to filter noise.
- **GraphRAG** [23]: It employs LLM for graph construction. When retrieval, it aggregates nodes into communities w.r.t. the query, and generates community reports to encapsulate global information from texts.
- **LightRAG** [24]: It calculates the vector similarity between query extracted entities and graph nodes, achieving a one-to-one mapping from entities to graph nodes, then retrieve these nodes' graph neighborhood.

Walking-based Models These models apply walking-based graph algorithms to score graph nodes, then retrieve text chunks w.r.t. their contained entity nodes' score.

- **F.GraphRAG** [32]: Fast-Graphrag employs LLM for graph construction. During retrieval, it applies PPR to assign node scores based on the query, and retrieve the Top-k passages based on the sum of their contained nodes' scores.
- **HippoRAG2** [29]: It employs LLM for graph construction, and also add the text chunks as nodes in the graph. As a result, it directly retrieves Top-k text chunks based on their respective nodes' PPR score.

*4) **Hyperparameter and Hardware Settings**:* We set the retrieval top-k value for all RAG models to 5, aligning with the settings of Fast-GraphRAG and HippoRAG2. For GraphRAG, we apply its local search mode to all tasks, as it achieves the overall best effectiveness. For LightRAG, we apply its hybrid

Table IV: Model efficiency on 4 datasets for 5 tasks, all experiments were carried out with Qwen2.5-14B-Instruct LLM backbone.

Category	Models	Time Cost (Indexing / Querying mins)				Token Cost (Input / Output per query)			
		Novel	Medical	WOS	IFS-REL	Novel	Medical	WOS	IFS-REL
Traditional RAG	NaiveRAG	**13** / 371	**7** / 245	**2** / 292	**2** / 225	9,437 / 1,708	7,473 / 379	7,292 / 29	4,521 / 16
	L.LLMLingua	51 / 360	37 / 305	4 / 358	3 / 315	8,466 / 1,750	5,686 / 390	5,861 / 31	2,757 / **14**
Tree -based RAG	RAPTOR	464 / 250	117 / 319	30 / 162	24 / **121**	7,243 / 777	4,817 / 695	6,007 / 41	3,807 / 33
	SiReRAG	85 / 233	40 / 207	26 / 139	18 / 117	7,759 / 800	4,918 / 701	6,012 / 47	3,821 / 36
Community -based RAG	G-Retriever	19 / **119**	12 / 145	8 / 158	3 / 164	6,344 / **116**	6,290 / **157**	6,762 / **12**	4,002 / 21
	MIXRAG	17 / 388	11 / 453	5 / 257	**2** / 193	**6,083** / 237	**4,238** / 230	4,193 / 31	3,597 / 29
	GraphRAG	347 / 205	194 / 158	85 / 240	64 / 217	12,686 / 2,109	8,385 / 1,017	11,809 / 351	9,009 / 287
	LightRAG	166 / 221	116 / 164	42 / 211	31 / 197	13,145 / 1,233	9,535 / 426	11,510 / 227	7,667 / 199
Walking -based RAG	HippoRAG2	95 / 134	46 / **130**	19 / 140	15 / 127	8,138 / 1,073	6,264 / 595	5,066 / 93	2,440 / 102
	F.GraphRAG	106 / 198	48 / 173	29 / 186	27 / 164	7,490 / 1,252	6,475 / 384	4,957 / 35	2,217 / 20
	AGRAG	65 / 142	16 / 138	16 / **132**	14 / 124	6,487 / 1,140	5,145 / 413	**3,970** / 101	**1,890** / 93

search mode for all tasks except the text classification task, as it is claimed as one of the contributions of LightRAG. For Fast-GraphRAG and HippoRAG2, we set their hyperparameters to default, to align with their original paper and code. Other hyperparameters of models were set as default by their codes.

For our AGRAG, the length of text chunks l_t is set to 256, and token overlap between chunks l_o to 32, the number of raw mapped triples k_a and the number of hybrid retrieved chunks k_r are both set to 5, to align with the setting of HippoRAG2 and GraphRAG-bench. AGRAG's unique hyperparameter, the entity extraction threshold, and the maximum n-gram, were set to 0.5 and 3, respectively. Please refer to Section IV-E for further parameter sensitivity analysis.

All our experiments were carried out and evaluated with Qwen2.5-14B-Instruct [83] LLM backbone deployed with vLLM [84] if not further illustrated. We apply the Contriever [85] embedding model for HippoRAG2 and AGRAG's dense passage retrieval procedure, and bge-large-en-v1.5 [86] embedding model for other baselines' LLM embedding. We apply Qwen-3-Reranker-0.6B LLM for raw triplet filtering. All experiments are conducted on an Intel(R) Xeon(R) Gold 5220R CPU and a single NVIDIA A100-80GB GPU.

B. Effectiveness Evaluation

Based on the experiment results in Table III we have observations as follow:

Firstly, in line with recent findings [31], [34], [35], NaiveRAG [15] achieves better performance than all tested walking-based baselines on Contextual Summarization and Creative Generation tasks. This is because, walking-based baselines overlook the global, and coarse-grained information that is essential for effective summarization and creative generation. AGRAG achieves the best performance on these tasks by providing the reasoning subgraph, as well as the hybrid retrieved text chunks to the LLM, offering more comprehensive coarse to fine-grained context for query answering.

Secondly, Our AGRAG achieves the best performance on Fact Retrieval and Complex Reasoning tasks. as it can benefit from phrase-level dependencies by generating explicit reasoning chains and directly feeding it to the LLM as guidance.

Lastly, in the text classification task, HippoRAG2 [29] and SiReRAG [71] achieve the second-best performance across all metrics on WOS and IFS-Rel dataset respectively, outperformed by AGRAG. This is because text classification requires capturing correlations between specific text entities and the text chunks to be classified, and all of SiReRAG, HippoRAG2 and AGRAG explicitly model entity-to-passage relationships in the KG. AGRAG achieves the best performance as its reasoning subgraph spans multiple entity nodes in the KG and explicitly captures the relationships between these entities and the text chunks associated with a given classification label, which can assist LLM classification.

Table V: The hallucinated prediction rate on WOS dataset.

Category	Model	Hallucinated Rate
Pure LLM	Qwen-2.5-14B	36.88%
Traditional RAG	NaiveRAG	9.96%
	L.LLMLingua	21.96%
Tree-based RAG	RAPTOR	7.40%
	SiReRAG	7.36%
Community-based RAG	GraphRAG	6.90%
	LightRAG	5.89%
Walking-based RAG	HippoRAG2	4.25%
	F.GraphRAG	6.06%
	AGRAG	**0.40%**

Furthermore, Table V presents the hallucinated prediction rate, defined as the percentage of classifications out of the label set. AGRAG achieves the lowest rate by leveraging MCMI subgraphs to provide explicit evidence for retrieved chunks.

C. Efficiency Evaluation

To evaluate the efficiency of AGRAG, we compare AGRAG's time and token cost with other baselines' on all 4 dataset tested, the results are shown in Table IV. According to the result, AGRAG can also achieve the best time and token efficiency compared with other baselines.

These results lead to two major findings: Firstly, both HippoRAG2 and AGRAG achieve superior time efficiency than other baselines, this suggests that frequent LLM calls, particularly those involving long input sequences, are the primary efficiency bottleneck in most Graph-based RAG models. By avoiding the LLM-based graph summarization process, HippoRAG2 and AGRAG significantly alleviate this bottleneck, leading to improved performance.

Secondly, although AGRAG introduces an additional procedure generating the MCMI reasoning graph, the statistics-based entity extraction method avoids the LLM calling. Denote

Table VI: The ablation experiment result on the two datasets and 4 tasks. All experiments were carried out with Qwen2.5-14B-Instruct LLM, where Time (I / Q) denotes the Index and Query time, Token (I / O) denotes the LLM Input and Output token count.

Model	Creative Gen.						Contextual Sum.				Complex Rea.				Fact Retrieval				Model Efficiency			
	Novel			Medical			Novel		Medical		Novel		Medical		Novel		Medical		Novel		Medical	
	FS.	ACC.	COV.	FS.	ACC.	COV.	COV.	ACC.	COV.	ACC.	ROG.	ACC.	ROG.	ACC.	ROG.	ACC.	ROG.	ACC.	Time (I / Q)	Token (I / G)	Time (I / Q)	Token (I / G)
AGRAG w. HR Only	0.491	0.322	0.378	0.303	0.526	0.549	0.739	0.522	0.535	0.583	0.303	0.453	0.334	0.636	0.323	0.487	0.355	0.619	**11.7** / 137.9	**5,117** / 193	11.4 / 117.9	5,215 / 199
AGRAG w. LLM ER	0.428	0.302	0.365	0.352	0.526	0.587	0.755	0.552	0.600	0.566	0.320	0.465	0.333	0.624	0.322	0.480	0.353	0.613	83.3 / 161.2	8,030 / 887	31.9 / 148.8	5,457 / 414
AGRAG w.o. HR	**0.519**	0.306	0.343	0.342	0.521	0.570	0.677	0.534	0.420	0.578	0.318	0.434	0.324	0.598	0.314	0.487	0.344	0.615	61.7 / 138.9	6,986 / 741	9.4 / **114.4**	**4,820** / 315
AGRAG w.o. MCMI	0.488	0.332	0.358	0.351	0.530	0.575	0.748	0.592	0.600	0.588	**0.324**	0.465	0.329	0.633	0.322	0.470	**0.359**	0.621	63.9 / **130.4**	6,710 / 757	9.5 / 125.0	5,230 / 331
AGRAG w.o. Cle.	0.478	0.311	0.361	0.336	0.517	0.569	0.712	0.597	0.616	0.577	0.301	0.450	0.334	0.628	0.313	0.478	0.354	0.620	65.6 / 140.1	6,988 / **154**	9.1 / 142.9	5,252 / **158**
AGRAG	0.513	**0.339**	**0.386**	**0.356**	**0.598**	**0.589**	**0.778**	**0.643**	**0.690**	**0.604**	0.319	**0.467**	**0.343**	**0.665**	**0.324**	**0.490**	0.357	**0.628**	65.6 / 142.0	6,988 / 639	**9.1** / 145.3	5,252 / 306

Table VII: Low resource experiment effectiveness with Qwen3-4B-Instruct-2507 LLM on 2080Ti GPU, where model with -L denotes low resource setting.

Task	Creative Gen.						Contextual Sum.				Complex Rea.				Fact Retrieval			
Dataset	Novel			Medical			Novel		Medical		Novel		Medical		Novel		Medical	
Metric	FS.	ACC.	COV.	FS.	ACC.	COV.	COV.	ACC.	COV.	ACC.	ROG.	ACC.	ROG.	ACC.	ROG.	ACC.	ROG.	ACC.
SiReRAG-L	0.324	0.412	0.299	0.421	0.758	0.475	0.647	0.673	0.536	0.738	0.260	0.426	0.308	0.698	0.271	0.416	0.343	0.661
HippoRAG2-L	0.433	0.481	0.335	0.425	0.755	0.433	0.696	0.657	0.522	0.731	0.280	0.507	0.298	0.681	0.288	0.513	0.338	0.654
AGRAG-L	**0.441**	**0.519**	**0.354**	**0.436**	**0.777**	**0.490**	**0.745**	**0.706**	**0.545**	**0.751**	**0.293**	**0.552**	**0.312**	**0.739**	**0.297**	**0.560**	**0.351**	**0.701**

Table VIII: Low resource experiment efficiency with Qwen3-4B-Instruct-2507 LLM on 2080Ti GPU, where -L denotes low resource setting.

Dataset	Novel				Medical			
	Time (mins)		AVG. Token		Time (mins)		AVG. Token	
Metric	Index	Query	In	Out	Index	Query	In	Out
SiReRAG-L	52.0	183.5	7,898	**1,003**	18.1	182.3	**5,113**	827
HippoRAG2-L	55.7	**131.2**	7,964	1,354	21.8	**162.4**	5,577	584
AGRAG-L	**44.7**	132.1	**6,625**	1,172	**14.4**	167.3	5,282	**534**

Table IX: Breakdown time cost (mins) of AGRAG on 4 datasets.

Dataset	Entity Extraction	Chunk Encoding	Triplet Filter	MCMI Generation	Hybrid Retrieval	QA
Novel	62.0	2.8	2.3	3.3	2.4	98.9
Medical	15.1	0.6	2.8	9.1	7.0	72.7
WOS	7.4	0.3	3.4	1.6	0.4	106.7
IFS-Rel	5.6	0.2	1.4	0.3	0.1	98.9

the number of nodes in graph as $|\mathcal{V}|$, the number of nodes in text chunk t as $|\mathcal{V}_t|$, the token length of text chunk t as l_t, the dimension of LLM hidden state is d_{llm}, for each test chunk, AGRAG's statistics-based entity extraction achieves $O(|\mathcal{V}|)$ time complexity, and the LLM entity extraction method achieves $O(d_{\text{llm}}(|\mathcal{V}_t| + l_t)^2)$ complexity. As d_{llm} is very large for LLM models (e.g., over 12,000 for GPT-4o [87]), and $|\mathcal{V}_t| + l_t$ can also be a large number, our AGRAG can still achieve better overall efficiency.

Lastly, according to Table VII and Table VIII, where we test AGRAG and 2 other best performing baselines with VLLM [84] deployed Qwen3-4B-Instruct-2507 [88] LLM on two 2080TI GPUs. AGRAG-L still can achieve comparable efficiency under low resource setting, while also achieves better accuracy than normal. This is because the Qwen3-4B is a more novel LLM compared with Qwen2.5-14B, it contains richer general knowledge and can perform better in commonsense reasoning [88]. However, due to the model size limitation, Qwen2.5-14B is still better at context understanding and instruction following, making our normal setting outperform the low-resource setting in other metrics.

D. Ablation Study

To analyze AGRAG's performance, we conduct an extensive ablation study on GraphRAG-bench tested across 2 datasets and 4 tasks, the ablation models tested are listed as follow:

- **AGRAG w. HR Only**: Here, we apply only AGRAG's hybrid retrieval step, without any reasoning-path generation.
- **AGRAG w. LLM ER**: Here, we substitute AGRAG's statistics-based entity extraction method for HippoRAG2's LLM entity extraction approach to test its effects.
- **AGRAG w.o. HR**: Here, we remove the hybrid retrieved chunks in AGRAG's LLM input, and only input the reasoning subgraph and its involved text chunk to LLM.
- **AGRAG w.o. MCMI**: Here, we simply generate a minimum cost Steiner tree that contains all mapped fact triples as reasoning path, ignoring more complex reasoning structures.
- **AGRAG w.o. Cle.**: Here, we remove the query aware raw triplet filter procedure (Equation (9)) when graph retrieval.

Based on the results in Table VI, we can have the following observations regarding AGRAG:

Firstly, AGRAG w. HR Only achieves the best indexing time efficiency, as it omit all graph related procedures. However, the lack of fine-grained information from graph make it suffer from sub-optimal performance especially on Creative Generation and Contextual Summarization tasks that require summarization to fine-grained information.

Secondly, although AGRAG w. LLM ER is more time and token-consuming, it still achieves worse performance compared to AGRAG. This is because the LLM entity extraction approach can produce hallucinated outputs that fail to conform to the expected entity extraction format, leading to decoding errors. This highlights the importance of substituting the LLM entity extraction method to avoid hallucination.

Thirdly, AGRAG w.o. HR experiences a significant performance drop on the Contextual Summarization task and in the coverage score of the Creative Generation task. This is because it provides the LLM with only the reasoning path and its associated text chunks—while efficient, this limits the amount of supportive content available and restricts the model's ability to find sufficient grounded information.

Fourthly, AGRAG w.o. MCMI performs similarly to AGRAG on the Fact Retrieval and Complex Reasoning tasks but consistently underperforms AGRAG on the more challenging Contextual Summarization and Creative Generation tasks. This is because its simple, tree-structured reasoning paths are insufficient to support queries from harder tasks, which may require the inclusion of cyclic structures or multiple answer

nodes that are difficult to jointly capture within the Steiner tree framework.

Lastly, AGRAG w.o. Cle. achieves suboptimal performance compared to AGRAG, especially in more complex Creative Generation and Contextual Summarization tasks, which require summarization to graph knowledge. In these scenarios, AGRAG needs the filter procedure to ensure the query-relatedness of retrieved triples, otherwise noise triples are included and affect the model summarization. Considering that omitting the query-related triplet filter can only marginally improve AGRAG's efficiency (reduce less than 1% of time cost), we consider the triplet cleaning procedure to be worthwhile.

We also show the breakdown time cost in Table IX, where the main time efficiency bottleneck of AGRAG is the LLM QA procedure, which were inevitably suffered by all LLM-based RAG methods, other procedures only lead to a marginal time cost increase. On Novel dataset, although the entity extraction procedure achieve higher time cost, according to Table VI, this cost can be even higher if we apply the LLM based entity extraction approach, because AGRAG w. LLM ER (which applies LLM entity extraction method) achieves higher time and token cost than AGRAG.

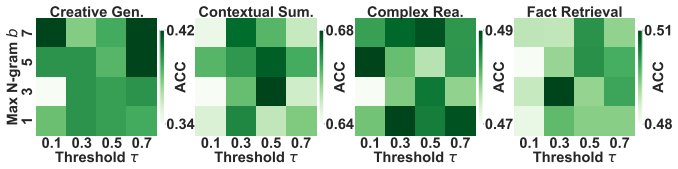
E. Parameter Sensitivity Analysis

In this subsection, we analyse the parameter sensitivity of AGRAG's two unique hyperparameters: The maximum n-gram b and the entity extraction threshold τ. As all other hyperparameters of AGRAG also exist in other baselines, we just align them with other baselines for fair comparison. We carry out the analysis on GraphRAG-bench [34], which cover most of the tasks of our experiment.

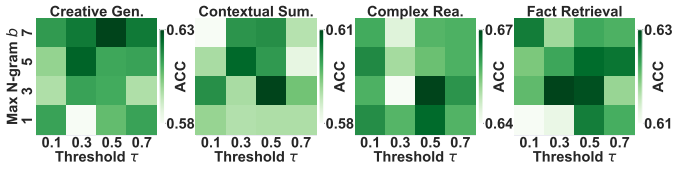
1) The maximum n-gram b: As introduced in Section III-B, the maximum n-gram b is used as the entity boundary to control the maximum allowed word length of the extracted entities. Here, we conduct a test about the maximum n-gram parameter $b \in \{1, 3, 5, 7\}$, as entity distribution in English is, in general, less than 7 words according to [89]. From the experiment results in Figure 3, AGRAG achieves the overall best performance when the maximum n-gram b is set to 3, this is because most of the entities usually consist of no more than 3 words, according to the discovery of [89]–[91].

For Creative Generation task, AGRAG can achieve better performance with larger maximum n-gram, because the correct answer of this task is more flexible than other tasks that have fixed answers or require answers to cover certain facts. Therefore, the negative impact of noise introduced by high n-gram phrases is overwhelmed by the additional context they provide, leading to better performance. This also highlights AGRAG's adaptability to different tasks through its flexible graph construction. In our experiments, we set the maximum n-gram b to 3 for all tasks.

2) The entity extraction threshold τ: As introduced in Section III-B, in the extraction procedure, we extract words or phrases that have the entity extraction score $\text{ER}(\cdot) > \tau$ as an entity, and we have $0 < \text{ER}(\cdot) < 1$. Considering the computational cost, we only test the entity extraction threshold



(a) Parameter sensitivity experiment on Novel dataset of GraphRAG-bench.



(b) Parameter sensitivity experiment on Medical dataset of GraphRAG-bench.

Figure 3: Parameter sensitivity experiment on GraphRAG-bench, where x-axis of each heatmap denotes the entity extraction threshold, and y-axis denotes the maximum n-gram. We choose accuracy as the metric; the darker the color of each block, the better the accuracy of the corresponding parameter pair.

$\tau \in \{0.1, 0.3, 0.5, 0.7\}$. According to the experiment results in Figure 3, AGRAG typically achieves its best performance when set to 0.3 or 0.5. This is because a large threshold tends to extract less informative entities, and a small one tends to cause insufficient entity extraction; both can affect the model's effectiveness. In our experiments, we set the entity extraction threshold τ to 0.5 for all tasks.

V. Conclusion

This paper presents AGRAG, an Advanced Graph-based Retrieval Augmented Generation framework addressing the Inaccurate Graph Construction, Poor Reasoning Ability, and Inadequate Answer issues of current RAG frameworks. By providing explicit reasoning paths to help LLM inference, AGRAG makes the LLM better focus on query-related contexts within the retrieved context. Experiments on real-world datasets demonstrate the effectiveness and efficiency of AGRAG over state-of-the-art RAG models.

VI. Acknowledgements

Lei Chen is supported by National Key Research and Development Program of China Grant No. 2023YFF0725100, National Science Foundation of China under Grant No. U22B2060, Guangdong-Hong Kong Technology Innovation Joint Funding Scheme Project No. 2024A0505040012, AOE Project AoE/E-603/18, Theme-based project TRS T41-603/20R, CRF Project C2004-21G, Key Areas Special Project of Guangdong Provincial Universities 2024ZDZX1006, Guangdong Province Science and Technology Plan Project 2023A0505030011, HKUST(GZ) CMCC(Guangzhou Branch) Metaverse Joint Innovation Lab under Grant No. P00659, Hong Kong ITC TC-SKLCRCC26EG01, ITF grant PRP/004/22FX, Zhujiang scholar program 2021JC02X170, HKUST Webank joint research lab. Haoyang Li's work is partially supported by grant No. PolyU P0052504, PolyU-Huawei P0053707, PolyU-Minshang P0057770, PolyU TDG25-28/TBP/11-IICA.

VII. AI-Generated Content Acknowledgement

In this paper, we use AI models: Qwen3-Max and Gemini3, to polish writing as well as code debugging.

References

[1] Y. Sun, S. Wang, S. Feng, S. Ding, C. Pang, J. Shang, J. Liu, X. Chen, Y. Zhao, Y. Lu et al., "Ernie 3.0: Large-scale knowledge enhanced pre-training for language understanding and generation," arXiv preprint arXiv:2107.02137, 2021.

[2] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier et al., "Mistral 7b," arXiv preprint arXiv:2310.06825, 2023.

[3] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar et al., "Llama: Open and efficient foundation language models," arXiv preprint arXiv:2302.13971, 2023.

[4] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat et al., "Gpt-4 technical report," arXiv preprint arXiv:2303.08774, 2023.

[5] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou, C. Li, C. Li, D. Liu, F. Huang et al., "Qwen2 technical report," arXiv preprint arXiv:2407.10671, 2024.

[6] C. Zhang, Y. Mao, Y. Fan, Y. Mi, Y. Gao, L. Chen, D. Lou, and J. Lin, "Finsql: Model-agnostic llms-based text-to-sql framework for financial analysis," in Companion of the 2024 International Conference on Management of Data, 2024, pp. 93–105.

[7] D. Chen, Y. Huang, Z. Ma, H. Chen, X. Pan, C. Ge, D. Gao, Y. Xie, Z. Liu, J. Gao et al., "Data-juicer: A one-stop data processing system for large language models," in Companion of the 2024 International Conference on Management of Data, 2024, pp. 120–134.

[8] K. Lu, Z. Yang, S. Wu, R. Xia, D. Zhang, and G. Chen, "Adda: Towards efficient in-database feature generation via llm-based agents," Proceedings of the ACM on Management of Data, vol. 3, no. 3, pp. 1–27, 2025.

[9] S. Jo and I. Trummer, "Sparellm: Automatically selecting task-specific minimum-cost large language models under equivalence constraint," Proceedings of the ACM on Management of Data, vol. 3, no. 3, pp. 1–26, 2025.

[10] M. I. L. Balaka, D. Alexander, Q. Wang, Y. Gong, A. Krisnadhi, and R. Castro Fernandez, "Pneuma: Leveraging llms for tabular data representation and retrieval in an end-to-end system," Proceedings of the ACM on Management of Data, vol. 3, no. 3, pp. 1–28, 2025.

[11] W. Zhou, Y. Gao, X. Zhou, and G. Li, "Cracking sql barriers: An llm-based dialect translation system," Proceedings of the ACM on Management of Data, vol. 3, no. 3, pp. 1–26, 2025.

[12] J. S. Barbosa, U. Gondhali, G. Petrossian, K. Sharma, S. Chakraborty, J. Jacquet, and J. Freire, "A cost-effective llm-based approach to identify wildlife trafficking in online marketplaces," Proceedings of the ACM on Management of Data, vol. 3, no. 3, pp. 1–23, 2025.

[13] K. Luoma and A. Kumar, "Snails: Schema naming assessments for improved llm-based sql inference," Proceedings of the ACM on Management of Data, vol. 3, no. 1, pp. 1–26, 2025.

[14] R. Omar, O. Mangukiya, and E. Mansour, "Dialogue benchmark generation from knowledge graphs with cost-effective retrieval-augmented llms," Proceedings of the ACM on Management of Data, vol. 3, no. 1, pp. 1–26, 2025.

[15] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv preprint arXiv:2312.10997, 2023.

[16] B. Peng, Y. Zhu, Y. Liu, X. Bo, H. Shi, C. Hong, Y. Zhang, and S. Tang, "Graph retrieval-augmented generation: A survey," arXiv preprint arXiv:2408.08921, 2024.

[17] S. Wu, Y. Xiong, Y. Cui, H. Wu, C. Chen, Y. Yuan, L. Huang, X. Liu, T.-W. Kuo, N. Guan et al., "Retrieval-augmented generation for natural language processing: A survey," arXiv preprint arXiv:2407.13193, 2024.

[18] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," Transactions of the Association for Computational Linguistics, vol. 12, pp. 157–173, 2024.

[19] T. Merth, Q. Fu, M. Rastegari, and M. Najibi, "Superposition prompting: improving and accelerating retrieval-augmented generation," in Proceedings of the 41st International Conference on Machine Learning, 2024, pp. 35 507–35 527.

[20] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li, "A survey on rag meeting llms: Towards retrieval-augmented large language models," in Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining, 2024, pp. 6491–6501.

[21] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel et al., "Retrieval-augmented generation for knowledge-intensive nlp tasks," Advances in neural information processing systems, vol. 33, pp. 9459–9474, 2020.

[22] H. Han, Y. Wang, H. Shomer, K. Guo, J. Ding, Y. Lei, M. Halappanavar, R. A. Rossi, S. Mukherjee, X. Tang et al., "Retrieval-augmented generation with graphs (graphrag)," arXiv preprint arXiv:2501.00309, 2024.

[23] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph rag approach to query-focused summarization," arXiv preprint arXiv:2404.16130, 2024.

[24] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, "Lightrag: Simple and fast retrieval-augmented generation," in Proceedings of the 2025 conference on empirical methods in natural language processing, 2025.

[25] Z. Li, X. Chen, H. Yu, H. Lin, Y. Lu, Q. Tang, F. Huang, X. Han, L. Sun, and Y. Li, "Structrag: Boosting knowledge intensive reasoning of llms via inference-time hybrid information structurization," in The Thirteenth International Conference on Learning Representations.

[26] Y. Hu, Z. Lei, Z. Zhang, B. Pan, C. Ling, and L. Zhao, "Grag: Graph retrieval-augmented generation," 2024. [Online]. Available: https://arxiv.org/abs/2405.16506

[27] B. Jimenez Gutierrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "Hipporag: Neurobiologically inspired long-term memory for large language models," Advances in Neural Information Processing Systems, vol. 37, pp. 59 532–59 569, 2024.

[28] L. Liang, Z. Bo, Z. Gui, Z. Zhu, L. Zhong, P. Zhao, M. Sun, Z. Zhang, J. Zhou, W. Chen et al., "Kag: Boosting llms in professional domains via knowledge augmented generation," in Companion Proceedings of the ACM on Web Conference 2025, 2025, pp. 334–343.

[29] B. J. Gutiérrez, Y. Shu, W. Qi, S. Zhou, and Y. Su, "From rag to memory: Non-parametric continual learning for large language models," Forty-second International Conference on Machine Learning, 2025.

[30] S. Wang, Y. Fang, Y. Zhou, X. Liu, and Y. Ma, "Archrag: Attributed community-based hierarchical retrieval-augmented generation," arXiv preprint arXiv:2502.09891, 2025.

[31] Y. Zhou, Y. Su, Y. Sun, S. Wang, T. Wang, R. He, Y. Zhang, S. Liang, X. Liu, Y. Ma et al., "In-depth analysis of graph-based rag in a unified framework," Proceedings of the VLDB Endowment, vol. 18, no. 13, pp. 5623–5637, 2025.

[32] circlemind ai. (2025) fast-graphrag. [Online]. Available: https://github.com/circlemind-ai/fast-graphrag

[33] T. H. Haveliwala, "Topic-sensitive pagerank," in Proceedings of the 11th international conference on World Wide Web, 2002, pp. 517–526.

[34] Z. Xiang, C. Wu, Q. Zhang, S. Chen, Z. Hong, X. Huang, and J. Su, "When to use graphs in rag: A comprehensive analysis for graph retrieval-augmented generation," arXiv preprint arXiv:2506.05690, 2025.

[35] H. Han, H. Shomer, Y. Wang, Y. Lei, K. Guo, Z. Hua, B. Long, H. Liu, and J. Tang, "Rag vs. graphrag: A systematic evaluation and key insights," arXiv preprint arXiv:2502.11371, 2025.

[36] C. Sammut and G. I. Webb, Eds., TF–IDF. Boston, MA: Springer US, 2010, pp. 986–987. [Online]. Available: https://doi.org/10.1007/978-0-387-30164-8_832

[37] S. Robertson, H. Zaragoza et al., "The probabilistic relevance framework: Bm25 and beyond," Foundations and Trends® in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[38] R. C. Fernandez, A. J. Elmore, M. J. Franklin, S. Krishnan, and C. Tan, "How large language models will disrupt data management," pp. 3302–3309, 2023. [Online]. Available: https://www.vldb.org/pvldb/vol16/p3302-fernandez.pdf

[39] S. Amer-Yahia, A. Bonifati, L. Chen, G. Li, K. Shim, J. Xu, and X. Yang, "From large language models to databases and back: A discussion on research and education," SIGMOD Rec., vol. 52, no. 3, pp. 49–56, 2023. [Online]. Available: https://doi.org/10.1145/3631504.3631518

[40] M. Zhang, Z. Ji, Z. Luo, Y. Wu, and C. Chai, "Applications and challenges for large language models: From data management perspective," in 40th IEEE International Conference on Data Engineering, ICDE 2024, Utrecht, The Netherlands, May 13-16, 2024. IEEE, 2024, pp. 5530–5541. [Online]. Available: https://doi.org/10.1109/ICDE60146.2024.00441

[41] X. Miao, Z. Jia, and B. Cui, "Demystifying data management for large language models," in Companion of the 2024 International

Conference on Management of Data, SIGMOD/PODS 2024, Santiago AA, Chile, June 9-15, 2024, P. Barceló, N. Sánchez-Pi, A. Meliou, and S. Sudarshan, Eds. ACM, 2024, pp. 547–555. [Online]. Available: https://doi.org/10.1145/3626246.3654683

[42] B. Snyder, M. Moisescu, and M. B. Zafar, "On early detection of hallucinations in factual question answering," in Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 2721–2732.

[43] A. Zhong, D. Mo, G. Liu, J. Liu, Q. Lu, Q. Zhou, J. Wu, Q. Li, and Q. Wen, "Logparser-llm: Advancing efficient log parsing with large language models," in Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 4559–4570.

[44] M. Yan, Y. Wang, K. Pang, M. Xie, and J. Li, "Efficient mixture of experts based on large language models for low-resource data preprocessing," in Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 3690–3701.

[45] L. Chen, F. Xu, N. Li, Z. Han, M. Wang, Y. Li, and P. Hui, "Large language model-driven meta-structure discovery in heterogeneous information network," in Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 307–318.

[46] Y. Ding, C. Niu, F. Wu, S. Tang, C. Lyu, and G. Chen, "Enhancing on-device llm inference with historical cloud-based llm interactions," in Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 597–608.

[47] S. Arora, B. Yang, S. Eyuboglu, A. Narayan, A. Hojel, I. Trummer, and C. Ré, "Language models enable simple systems for generating structured views of heterogeneous data lakes," Proceedings of the VLDB Endowment, vol. 17, no. 2, pp. 92–105, 2023.

[48] Y. Dong, C. Xiao, T. Nozawa, M. Enomoto, and M. Oyamada, "Deepjoin: Joinable table discovery with pre-trained language models," Proceedings of the VLDB Endowment, vol. 16, no. 10, pp. 2458–2470, 2023.

[49] M. Kayali, A. Lykov, I. Fountalis, N. Vasiloglou, D. Olteanu, and D. Suciu, "Chorus: Foundation models for unified data discovery and exploration," Proceedings of the VLDB Endowment, vol. 17, no. 8, pp. 2104–2114, 2024.

[50] Y. Zhang, A. Floratou, J. Cahoon, S. Krishnan, A. C. Müller, D. Banda, F. Psallidas, and J. M. Patel, "Schema matching using pre-trained language models," in 2023 IEEE 39th International Conference on Data Engineering (ICDE). IEEE, 2023, pp. 1558–1571.

[51] J. Tu, J. Fan, N. Tang, P. Wang, G. Li, X. Du, X. Jia, and S. Gao, "Unicorn: A unified multi-tasking model for supporting matching tasks in data integration," Proceedings of the ACM on Management of Data, vol. 1, no. 1, pp. 1–26, 2023.

[52] M. Fan, X. Han, J. Fan, C. Chai, N. Tang, G. Li, and X. Du, "Cost-effective in-context learning for entity resolution: A design space exploration," in 2024 IEEE 40th International Conference on Data Engineering (ICDE). IEEE, 2024, pp. 3696–3709.

[53] X. Du, G. Yuan, S. Wu, G. Chen, and P. Lu, "In situ neural relational schema matcher," in 2024 IEEE 40th International Conference on Data Engineering (ICDE). IEEE, 2024, pp. 138–150.

[54] B. Li, Y. Luo, C. Chai, G. Li, and N. Tang, "The dawn of natural language to SQL: are we fully ready? [experiment, analysis \u0026 benchmark]," Proc. VLDB Endow., vol. 17, no. 11, pp. 3318–3331, 2024. [Online]. Available: https://www.vldb.org/pvldb/vol17/p3318-luo.pdf

[55] T. Ren, Y. Fan, Z. He, R. Huang, J. Dai, C. Huang, Y. Jing, K. Zhang, Y. Yang, and X. S. Wang, "PURPLE: making a large language model a better SQL writer," in 40th IEEE International Conference on Data Engineering, ICDE 2024, Utrecht, The Netherlands, May 13-16, 2024. IEEE, 2024, pp. 15–28. [Online]. Available: https://doi.org/10.1109/ICDE60146.2024.00009

[56] I. Trummer, "From bert to gpt-3 codex: harnessing the potential of very large language models for data management," Proceedings of the VLDB Endowment, vol. 15, no. 12, pp. 3770–3773, 2022.

[57] Z. Gu, J. Fan, N. Tang, L. Cao, B. Jia, S. Madden, and X. Du, "Few-shot text-to-sql translation using structure and content prompt learning," Proceedings of the ACM on Management of Data, vol. 1, no. 2, pp. 1–28, 2023.

[58] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen et al., "Siren's song in the ai ocean: a survey on hallucination in large language models," arXiv preprint arXiv:2309.01219, 2023.

[59] T. Chen, H. Wang, S. Chen, W. Yu, K. Ma, X. Zhao, H. Zhang, and D. Yu, "Dense x retrieval: What retrieval granularity should we use?" arXiv preprint arXiv:2312.06648, 2023.

[60] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "Raptor: Recursive abstractive processing for tree-organized retrieval," arXiv preprint arXiv:2401.18059, 2024.

[61] Z. Hu, C. Wang, Y. Shu, H.-Y. Paik, and L. Zhu, "Prompt perturbation in retrieval-augmented generation based large language models," in Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 1119–1130.

[62] J. Wu, C.-C. Chang, T. Yu, Z. He, J. Wang, Y. Hou, and J. McAuley, "Coral: Collaborative retrieval-augmented large language models improve long-tail recommendation," in Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 3391–3401.

[63] T. Cai, Z. Tan, X. Song, T. Sun, J. Jiang, Y. Xu, Y. Zhang, and J. Gu, "Forag: Factuality-optimized retrieval augmented generation for web-enhanced long-form question answering," in Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 199–210.

[64] H. Oosterhuis, R. Jagerman, Z. Qin, X. Wang, and M. Bendersky, "Reliable confidence intervals for information retrieval evaluation using generative ai," in Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 2307–2317.

[65] S. Yadav, D. Saini, A. Buvanesh, B. Paliwal, K. Dahiya, S. Asokan, Y. Prabhu, J. Jiao, and M. Varma, "Extreme meta-classification for large-scale zero-shot retrieval," in Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 3657–3666.

[66] T.-Y. Che, X.-L. Mao, T. Lan, and H. Huang, "A hierarchical context augmentation method to improve retrieval-augmented llms on scientific papers," in Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2024, pp. 243–254.

[67] X. Zhou, X. Li, Y. Peng, M. Xu, X. Zhang, M. Yu, Y. Wang, X. Jia, K. Wang, Q. Wen et al., "Emorag: Evaluating rag robustness to symbolic perturbations," arXiv preprint arXiv:2512.01335, 2025.

[68] Z. Wang, H. Yuan, W. Dong, G. Cong, and F. Li, "Corag: A cost-constrained retrieval optimization system for retrieval-augmented generation," arXiv preprint arXiv:2411.00744, 2024.

[69] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Ruhle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and D. Zhang, "Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression," in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, aug 2024, pp. 963–981. [Online]. Available: https://aclanthology.org/2024.findings-acl.57

[70] H. Jiang, Q. Wu, , X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, "Longllmlingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression," in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, aug 2024, pp. 1658–1677. [Online]. Available: https://aclanthology.org/2024.acl-long.91

[71] N. Zhang, P. K. Choubey, A. Fabbri, G. Bernadett-Shapiro, R. Zhang, P. Mitra, C. Xiong, and C.-S. Wu, "SireRAG: Indexing similar and related information for multihop reasoning," in The Thirteenth International Conference on Learning Representations, 2025. [Online]. Available: https://openreview.net/forum?id=yp95goUAT1

[72] X. He, Y. Tian, Y. Sun, N. Chawla, T. Laurent, Y. LeCun, X. Bresson, and B. Hooi, "G-retriever: Retrieval-augmented generation for textual graph understanding and question answering," Advances in Neural Information Processing Systems, vol. 37, pp. 132 876–132 907, 2024.

[73] L. Liu and C. J. Yang, "Mixrag: Mixture-of-experts retrieval-augmented generation for textual graph understanding and question answering," arXiv preprint arXiv:2509.21391, 2025.

[74] L. Luo, Y.-F. Li, G. Haffari, and S. Pan, "Reasoning on graphs: Faithful and interpretable large language model reasoning," in International Conference on Learning Representations, 2024.

[75] D. Zhou, Y. Zhu, X. Wang, H. Zhou, Y. He, J. Chen, S. Staab, and E. Kharlamov, "What breaks knowledge graph based rag? empirical insights into reasoning under incomplete knowledge," arXiv preprint arXiv:2508.08344, 2025.

[76] P. Boldi, M. Santini, and S. Vigna, "Pagerank as a function of the damping factor," in Proceedings of the 14th international conference on World Wide Web, 2005, pp. 557–566.

[77] R. Baeza-Yates, P. Boldi, and C. Castillo, "Generic damping functions for propagating importance in link-based ranking," Internet Mathematics, vol. 3, no. 4, pp. 445–478, 2006.

[78] K. Mehlhorn, "A faster approximation algorithm for the steiner problem in graphs," Information Processing Letters, vol. 27, no. 3, pp. 125–128, 1988.

[79] K. Kowsari, D. E. Brown, M. Heidarysafa, K. J. Meimandi, M. S. Gerber, and L. E. Barnes, "Hdltex: Hierarchical deep learning for text classification," in 2017 16th IEEE international conference on machine learning and applications (ICMLA). IEEE, 2017, pp. 364–371.

[80] C. Xia, W. Yin, Y. Feng, and P. S. Yu, "Incremental few-shot text classification with multi-round new classes: Formulation, dataset and system," in Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021, K. Toutanova, A. Rumshisky, L. Zettlemoyer, D. Hakkani-Tür, I. Beltagy, S. Bethard, R. Cotterell, T. Chakraborty, and Y. Zhou, Eds. Association for Computational Linguistics, 2021, pp. 1351–1360. [Online]. Available: https://doi.org/10.18653/v1/2021.naacl-main.106

[81] VibrantLabs, "Ragas: Supercharge your llm application evaluations," https://github.com/vibrantlabsai/ragas, 2024.

[82] C.-Y. Lin, "Rouge: A package for automatic evaluation of summaries," in Text Summarization Branches Out. Barcelona, Spain: Association for Computational Linguistics, jul 2004, pp. 74–81. [Online]. Available: https://aclanthology.org/W04-1013/

[83] Qwen, :, A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu, "Qwen2.5 technical report," 2025. [Online]. Available: https://arxiv.org/abs/2412.15115

[84] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles, 2023.

[85] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, "Unsupervised dense information retrieval with contrastive learning," Transactions on Machine Learning Research.

[86] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-pack: Packaged resources to advance general chinese embedding," 2023.

[87] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford et al., "Gpt-4o system card," arXiv preprint arXiv:2410.21276, 2024.

[88] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv et al., "Qwen3 technical report," arXiv preprint arXiv:2505.09388, 2025.

[89] J. F. Silva and J. C. Cunha, "An empirical model for n-gram frequency distribution in large corpora," in Pacific-Asia Conference on Knowledge Discovery and Data Mining. Springer, 2020, pp. 840–851.

[90] Q. Zhu, X. Li, A. Conesa, and C. Pereira, "Gram-cnn: a deep learning approach with local context for named entity recognition in biomedical text," Bioinformatics, vol. 34, no. 9, pp. 1547–1554, 2018.

[91] I. Fette, "Combining n-gram based statistics with traditional methods for named entity recognition," 2007.